

Caso pedagógico 02 — Falha de autenticação no Postgres e migrações não aplicadas

Contexto

Em aplicações com banco de dados (especialmente em ambientes containerizados), pequenas inconsistências de configuração podem causar efeitos em cascata: serviços entrando em *crash loop*, migrações não aplicadas, tabelas inexistentes e uma sensação de “ambiente instável” que atrapalha testes, integração e — no nosso caso — atividades de aula.

Este caso pedagógico aborda duas causas comuns e frequentemente correlacionadas: 1) **falha de autenticação no Postgres por variáveis de ambiente inconsistentes**; 2) **migrações não aplicadas por execução incorreta do data-source no container**.

Problema identificado

1) Credenciais inconsistentes no Compose

Havia divergência no nome da variável de senha do banco: - Arquivo de produção (`docker-compose.prod.yml`) usando `DB_PASSWORD`. - Código (TypeORM) lendo `process.env.DB_PASS`.

Além disso, em **dev**, existia um volume persistido do Postgres criado com uma senha antiga. Mesmo que o `.env` fosse corrigido, o banco já inicializado **continuará com a senha antiga**, pois o Postgres define as credenciais iniciais na primeira criação do volume.

2) Migrações falhando por data-source incorreto

Ao executar migrações do `users-service` a partir do container, o comando falhava porque: - o **data-source compilado** (em `dist/...`) não estava sendo usado; - o comando tentava referenciar caminhos de `src/` ou um data-source não compatível com o ambiente de execução do container.

Sintomas observáveis (o que o aluno enxerga)

- Serviços reiniciando com mensagens de autenticação (senha incorreta / conexão recusada).
 - Erros de banco do tipo:
`relation "users.users" does not exist`
 - Tabelas ausentes e *seeds* não aplicados.
 - Ambiente inconsistente, com falhas intermitentes.
-

Impacto potencial

1. Indisponibilidade em dev/prod

2. Serviços não sobem; API fica fora do ar.

3. Base em estado inválido

4. Schema incompleto; migrações não executadas; *seeds* ausentes.

5. Ambiente instável para testes e aulas

6. Alunos perdem tempo com problemas de infra, em vez de aprender o conteúdo-alvo.

7. Risco operacional

8. Em produção, divergências de variáveis podem gerar incidentes na troca de versão, *rollback* e escalonamento.

Solução fundamentada

A) Padronização de variáveis de ambiente

A solução foi **padronizar o nome da variável** para o que o código já utiliza: - `DB_PASS` (lida por `process.env.DB_PASS` no TypeORM)

Isso elimina ambiguidade e previne regressões quando múltiplos serviços compartilham a mesma convenção.

B) Reset controlado do volume em dev (quando permitido)

Em ambiente de desenvolvimento, quando é aceitável “perder dados”, o caminho mais direto é: - derrubar os containers e **remover volumes** (`down -v`) - subir novamente com a senha correta

Isso garante que o Postgres seja reinicializado com credenciais coerentes com o `.env`/compose.

Nota pedagógica importante: Em produção, **não** se remove volume; o caminho é rotação de senha, migrações seguras e gestão de segredos.

C) Migrações sempre antes de validar health

O *health check* só deve ser considerado “validado” após: 1) o banco aceitar conexão; 2) migrações aplicadas; 3) (se aplicável) seeds executados.

Caso contrário, o serviço pode até responder, mas falhará assim que executar a primeira query real.

D) Executar migrações com o data-source compilado

Como o container executa a aplicação compilada, o comando de migração deve apontar para o **data-source em** `dist/`, evitando divergência de paths e dependências de runtime.

Implementação (delta aplicado)

1) `docker-compose.prod.yml`

- Alterado `DB_PASSWORD` → `DB_PASS` em todos os serviços, alinhando com o código:
- TypeORM lê `process.env.DB_PASS`.

2) Reset do volume em dev (quando permitido)

Comandos:

```
docker compose -f docker-compose.dev.yml down -v  
  
docker compose -f docker-compose.dev.yml up -d
```

O objetivo aqui é recriar o volume com a senha `postgres` (ou a senha definida no ambiente) e remover qualquer estado antigo conflitante.

3) Migrações no `users-service` usando data-source compilado

Comando:

```
docker compose -f docker-compose.dev.yml run --rm users-service  
npx typeorm-ts-node-commonjs -d dist/app/src/data-source.js migration:run
```

Pontos críticos: - `run --rm` executa um container efêmero só para aplicar as migrações. - `-d dist/app/src/data-source.js` aponta explicitamente para o data-source compilado. - `migration:run` aplica as migrações pendentes.

Efeito prático (o que muda no comportamento do sistema)

Após o ajuste: - Serviços sobem com credenciais corretas. - Tabelas são criadas e *seeds* (quando existentes) são aplicados. - Erros de relação inexistente deixam de ocorrer. - O fluxo fica **documentado**, reduzindo reincidência. - O `events-service` passa a responder health **200** (observado via IPv4 no cenário descrito).

Checklist didático para validação em aula

Use este checklist para confirmar que a correção foi aplicada corretamente:

1. O Compose (prod/deploy) usa `DB_PASS` (não `DB_PASSWORD`).
2. O código lê `process.env.DB_PASS` (sem inconsistências).
3. Em dev, se permitido resetar dados:
4. foi executado `docker compose ... down -v`.
5. Postgres sobe e aceita conexão com a senha esperada.

6. Migrações foram executadas com o data-source compilado (`dist/...`).
 7. Erros `relation ... does not exist` não ocorrem mais.
 8. Health checks só são validados **após** migrações.
-

Perguntas para discussão (para consolidar aprendizado)

1. Por que alterar a senha no `.env` não corrige um Postgres já inicializado com volume persistente?
 2. Qual o risco de ter duas variáveis (`DB_PASS` e `DB_PASSWORD`) coexistindo no mesmo projeto?
 3. Por que o data-source em `dist/` é o mais apropriado para migração no contexto do container?
 4. Qual a ordem correta de validação do ambiente: subir serviços → migrations → seeds → health? Por quê?
 5. Como você faria isso em produção sem apagar dados?
-

Conclusão

Este caso mostra dois aprendizados operacionais essenciais: - **consistência de configuração** (nomes de variáveis padronizados) evita incidentes simples e caros; - **migração é parte do boot**: sem migrations, “subiu” não significa “está pronto”.

Ao corrigir a variável (`DB_PASS`), resetar volume em dev quando permitido e executar migrações usando o data-source compilado, o ambiente se torna previsível, reproduzível e mais adequado para desenvolvimento, testes e uso didático.